Explainable Fraud Intelligence Platform

Project Audit and Technical Documentation

Generated: 2026-10-10
Code revision audited: `1c235ca` on branch `claude/fair-sequence-fraud-detection-baiokj` (base `origin/Keerthan` @ `5a3a7c8`)
Follow-up pass: software commit `6c7eea9`, Experiment 3 commits `b58fa52` → `1c235ca` (this document describes the repository at the revision above)
Scope: software audit, repairs, validation and documentation; **not** a scientific validation
Data: all data in this project are synthetic

Contents

1. Executive summary 3
2. Problem statement and objectives 3
3. Technology stack (as found and installed) 4
4. Repository structure 4
5. System architecture 5
6. Data and feature engineering 6
7. Machine-learning models 7
8. API and database 7
9. Frontend and user workflow 8
10. Setup and execution 11
11. Audit findings 12
12. Completed fixes 14
13. Experimental results 14
14. Security and reliability 18
15. Limitations and future improvements 18
16. Final project status 19
Appendix A. API endpoint inventory 19
Appendix B. Feature dictionary 20
Appendix C. Model configuration 20
Appendix D. Test inventory 20
Appendix E. Glossary 21

1. Executive summary

What the project does. The platform builds a per-customer *Behavioral Fraud DNA* (nine features computed from the customer's own earlier transactions), passes the customer's previous ten transactions through an **LSTM** to obtain a temporal *Risk Score*, feeds that score and the nine features of the current transaction to a **random forest** that produces a *Fraud Score* (0–100), explains the result with **Tree SHAP**, compares the transaction with the customer's own history (*Behavioral Similarity*), stores every scored transaction in a database and shows everything in a React dashboard served by a FastAPI backend.

Problem addressed. Banks need fraud scores that adapt to each customer's normal behaviour and can be explained to an analyst. The project demonstrates that pipeline end to end on synthetic data.

Implementation status. The system runs end to end. At the current revision the backend suite gives **707 passed, 4 skipped (missing optional data), 0 failed**, the frontend has 13 tests (all pass), and the dashboard, scoring, explanation, batch upload, PDF report, fraud-ring view and model-performance view worked in real browser runs.

Major findings. The audit confirmed **11 defects** and fixed all of them (1 high, 5 medium, 5 low), the most serious being that a single request with `amount = Infinity` was accepted, stored and permanently corrupted that customer's features (F-01). It also listed **12 open items** (R-01 … R-12). The follow-up pass **fixed four** of them: the similarity score no longer invents a value for customers with fewer than 10 earlier transactions (R-05), PDF reports are built from the stored transaction instead of client-supplied scores (R-04), timestamps are limited to 2000–2100 (R-06) and the v1 data generator is reproducible (R-09); two more were partly fixed (R-11, R-12) and three were added (R-13 … R-15). **Nine items remain open**; the most important are that there is no authentication (R-01; requirements are documented, no scheme was chosen), the alert bands do not match the random forest's score scale (R-02) and per-request cost grows with history (R-03).

Evidence quality. The reported evaluation numbers are genuine: the v2 time-split metrics were recomputed from the committed raw scores and matched exactly; the hold-out headline numbers trace to a protocol-hashed report. **Experiment 3** (five new synthetic datasets, protocol committed before data existed) added three measurements: the Behavioral Similarity Score separates fraud from legitimate rows (ROC-AUC 0.866) but adds no measurable information beyond the classifier; the forest's score is close to calibrated on this generator; and the LSTM input adds detectable value over the same forest without it. These are **synthetic-data results** from a single generator and say nothing about real banking. Three things must be kept apart: *software correctness* (tests, probes, browser runs), *synthetic-data evaluation* (all model results) and *real-world validation* (none exists).

Most important remaining work. Decide how alerts are defined; choose and add authentication before any network exposure; obtain independent data (the main scientific gap); decide whether to adopt calibration in serving (Chapter 15).

2. Problem statement and objectives

Card and account fraud is rare (0.5–0.9 % of transactions in this data) and adversaries imitate normal behaviour, so a fixed rule such as "large amount" produces either many false alarms or missed fraud. Per-customer baselines make the same transaction look normal for one customer and suspicious for another. Analysts also need *reasons*, not only a number.

Objectives (from the project documents): model customer behaviour; estimate emerging risk from the recent sequence; classify the current transaction; explain each decision; show it interactively. **Intended users:** fraud analysts and reviewers of the project. **Scope:** a research and demonstration system on synthetic data. **Limits:** no real data, no authentication, no calibration in serving, single-process, demo-scale histories (Chapter 15).

3. Technology stack (as found and installed)

Layer	Technology (version verified in this audit)
Language runtime	Python 3.13.16 (README: 3.12+); Node.js 22.22.0, npm 10.9.4
API	FastAPI 0.139.0, Uvicorn 0.34.0, Pydantic 2.10.4
Database	SQLAlchemy 2.0.36; SQLite by default (`sqlite:///./fraud_platform.db`); `psycopg2-binary` 2.9.10 installed for PostgreSQL (documented as untested)
Machine learning	TensorFlow 2.21.0 with Keras 3.15.1 (LSTM, DNN); scikit-learn 1.8.0 (random forest); SHAP 0.52.0 (Tree SHAP); pandas 3.0.2, numpy 2.4.4
Reports	ReportLab 5.0.0 (per-transaction PDF)
Frontend	React 19.2, Vite 8.1, Tailwind CSS 4.3, Recharts 3.9, axios 1.20 (after the audit fix), oxlint
Tests	pytest ≥ 8, httpx, pypdf (backend: 37 modules, 711 tests collected); vitest (frontend: 13 tests)
Not present	Docker, CI configuration, authentication, a migration framework

4. Repository structure

```
fraud-intelligence-platform/
├── backend/
│   ├── app/main.py                FastAPI app: 11 endpoints, validation, batch, PDF
│   ├── app/schemas.py             request/response models, input limits
│   ├── app/inference_pipeline.py  scoring flow, in-memory histories, locking
│   ├── app/config.py              paths, CORS origins, sequence length (10)
│   ├── app/model_sets.py          which artifacts load (MODEL_SET), SHA-256 pins
│   ├── app/model_metadata.py      /model-info content
│   ├── app/report.py              per-transaction PDF (ReportLab)
│   ├── app/features/              feature_engineering.py, ground_truth.py
│   ├── app/models/                lstm, dnn, downstream_classifier, shap, similarity
│   ├── app/db/                    engine/session, Transaction model, migrations
│   ├── app/evaluation/            offline leakage-safe evaluation, protocols
│   ├── app/training/              candidate and multi-seed training
│   ├── models/saved/              previous production artifacts (v1)
│   ├── models/candidates*/        v2 artifacts; default = candidates_downstream/
│   │                              v2/seed_14/lstm_random_forest
│   ├── models/evaluation/         committed evaluation reports (JSON, scores)
│   └── tests/                     37 test modules
├── frontend/src/                  App.jsx, api.js, apiError.js, components/, severity.js, __tests__/
├── data/                          v1 generator + seed CSV; v2/ deterministic generator
├── docs/                          research notes, evaluation reports, protocols
├── test_data/batch_scoring/       three sample CSVs for the batch screen
├── docs/deployment-security-requirements.md   security and deployment requirements (no scheme chosen)
├── experiments/sequence_early_detection/   Experiment 2 documents (no results)
├── experiments/validation_exp3/   Experiment 3: protocol, scripts, results, independent-validation
note
└── reports/                       this audit (separate from the application)
```

Key modules.

File	Responsibility	Inputs → outputs	Depends on
`main.py`	HTTP layer, persistence, rollback on failure, batch parsing and limits	JSON/CSV → scored JSON, PDF	pipeline, db, schemas, report
`inference_pipeline.py`	rebuild features, run LSTM → classifier → SHAP → similarity, keep histories	one transaction dict → score dict	features, model_sets, models
`feature_engineering.py`	the nine features, past-only	raw rows → feature rows	`config.py`
`model_sets.py`	choose and verify artifacts, shapes, hashes	`MODEL_SET` → loaded models	config, features
`similarity.py`	z-score vs own history, exponential decay	feature vectors → 0–100	numpy
`shap_explainer.py`	SHAP values → readable reasons	normalised input → ≤4 reasons	shap
`evaluation/*`	offline evaluation (not in the request path)	datasets → JSON reports	data/v2

Documents under `docs/step*.md` are dated snapshots of earlier steps (for example "Stage C: NO PROMOTION" predates the Step 4D switch to the random forest); the README and `docs/model_selection_report.md` describe the current default.

5. System architecture

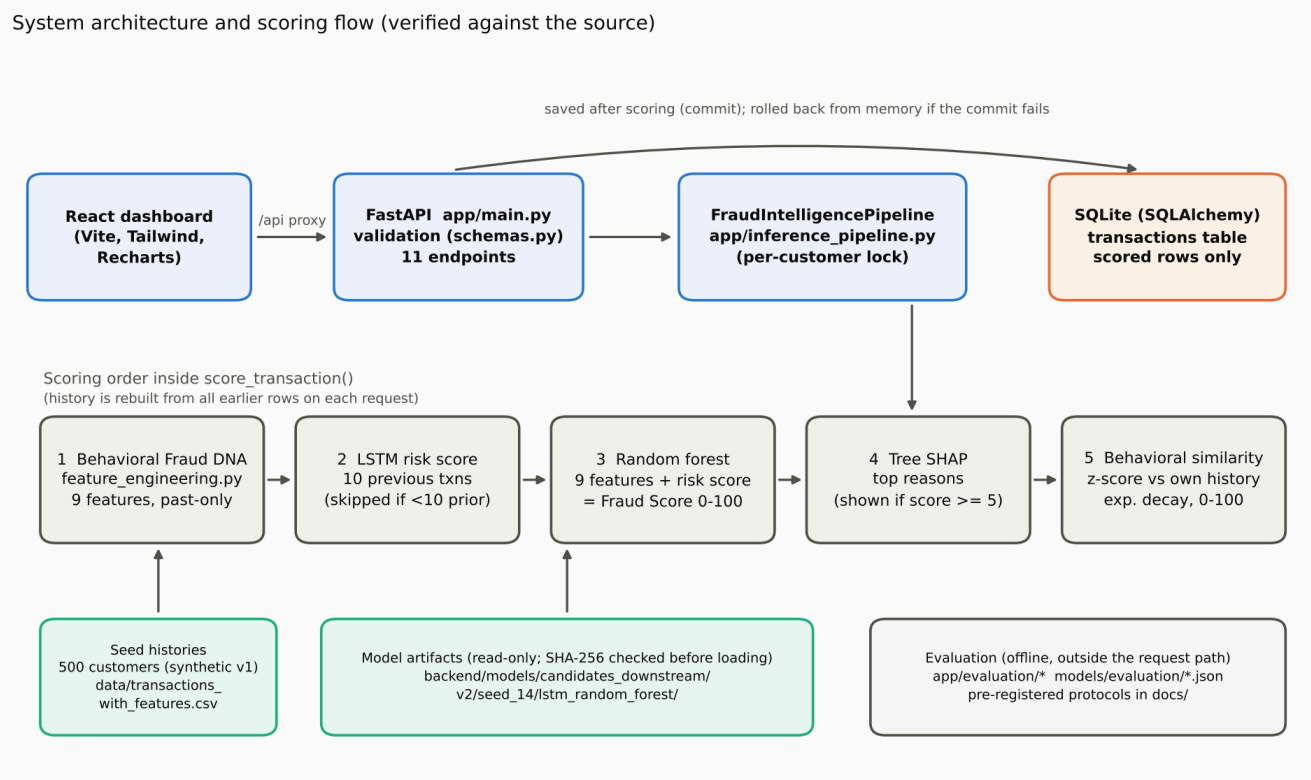


Figure 1. Components and scoring order, checked against `main.py` and `inference_pipeline.py`.

Boundaries and data flow. The browser talks only to `/api/*`, which Vite proxies to FastAPI on port 8000 (or `VITE_API_BASE_URL` plus CORS in production builds). `main.py` validates input, calls `FraudIntelligencePipeline.score_transaction` (serialised by a lock per process), then saves the result. The pipeline keeps each customer's history in memory, loaded from the v1 seed CSV and from the

database at start-up; the database holds **scored** transactions only. Models are loaded once at start-up.

Model execution order (inside `score_transaction`): (1) insert the new row in time order, recompute all features from the earlier rows; (2) LSTM on the previous ten rows (skipped with fewer than ten); (3) random forest on nine features + Risk Score (inputs scaled and clipped to ±6, score capped at 99.9); (4) alert level from fixed bands; (5) Tree SHAP reasons if the score is at least 5; (6) Behavioral Similarity against the mean and standard deviation of the earlier rows (only with at least 10 earlier transactions; otherwise it is withheld, status `insufficient_history`); (7) the history is replaced by the recomputed one. The database write happens afterwards in `main.py`; if it fails, the history change is undone (F-05).

6. Data and feature engineering

Datasets.

Dataset	Origin	Notes
v1 seed	`data/generate_synthetic_data.py` (500 customers, 180 days, ~12 % of customers get a "ramp-up week" plus a high-severity week, both labelled fraud)	93,913 rows, 819 fraud rows. Fraud rows use `DEV_UNKNOWN_*` device ids, foreign cities and high amounts; `amount_pct_of_avg` alone separates the classes perfectly (ROC-AUC 1.000). Used for dashboard histories and the `production` model set
v2	`data/v2/synth_v2` (7 fraud archetypes, rings, legitimate travel/new-device/VPN behaviour, login-failure events, explicit episodes)	deterministic: seed 42 regenerated here matched all committed SHA-256 hashes. Used to train and evaluate the default model set

Both are **synthetic**. No external dataset is used anywhere in the repository.

Raw schema (v1 features file, 17 columns): `customer_id, transaction_id, timestamp, amount, merchant_category, device_id, location, failed_logins_24h, is_new_device, is_new_location, is_fraud` plus the nine features below (v2 adds nine ground-truth metadata columns that are forbidden as model inputs; `features/ground_truth.py` enforces this at import time).

The nine features (`feature_engineering.build_point_features`; every statistic uses only earlier rows of the same customer; the history is updated after the features of a row are computed):

Feature	Definition
`amount_zscore`	(amount − mean of earlier amounts) / max(std of earlier amounts, 0.1 × mean, 1), clipped to ±10; with fewer than two earlier rows the std is taken as 0.3 × mean
`hour_is_unusual`	1 if fewer than 5 % of earlier transactions fell in this hour
`is_new_device`, `is_new_location`	1 if the device / city was not used in any earlier transaction
`is_foreign_location`	1 if the city is not one of 10 hard-coded Indian cities
`failed_logins_24h`	supplied with the transaction
`category_is_unusual`	1 if fewer than 5 % of earlier transactions were in this merchant category
`txn_velocity_1h`	number of earlier transactions within one hour of this one
`amount_pct_of_avg`	100 × amount / mean of earlier amounts, clipped to [0, 1000]

Temporal assumptions and limitations. No lookahead was found in the feature code (a perturbation-style check of future rows was not part of this audit; the project's own tests cover past-only windows). `is_foreign_location` depends on a hard-coded city list, so it is tied to this generator.

Features for a customer's first transaction have no baseline and are imputed to the training mean. The committed v1 CSV was generated by an earlier version of this code (R-08), so its stored values differ from a current recomputation in 43,442 rows.

7. Machine-learning models

Component	Verified implementation
LSTM risk predictor	input 10 × 9 normalised features; `Masking → LSTM 64 → Dropout 0.3 → LSTM 32 → Dropout 0.2 → Dense 16 → Dense 1 (sigmoid)`; 31,905 parameters; binary cross-entropy, Adam 1e-3, class weights. **Target: is the next transaction after the window fraudulent** (the window excludes the current one). Output × 100 = Risk Score. It is a recent-history risk signal, **not** shown to predict fraud before it happens (the project's own evaluation does not support that)
Random forest (default)	200 trees, depth 12, min leaf 5, no class weights; inputs = 9 features + Risk Score (10), scaled with saved means/stds and clipped to ±6; trained on **out-of-fold** LSTM scores; output = Fraud Score, capped at 99.9
DNN (previous default)	`64 → BatchNorm → Dropout → 32 → Dropout → 16 → 1`, 3,585 parameters, same 10 inputs
SHAP	exact interventional Tree SHAP on the forest's own score with a 200-row background; the four largest **positive** contributions become "reasons"; shown only if the score is ≥ 5; inputs imputed for cold start are never reported. SHAP describes the model's score; it does not establish causes
Behavioral Similarity	z = (current features − mean of earlier features) / std (std ≤ 1e-6 replaced by 1); `similarity = 100 · exp(−mean(abs(z)) / 2.9)`; deviation = 100 − similarity. All nine features weigh equally. **With fewer than 10 earlier transactions it is not computed**: `similarity_pct` and `deviation_pct` are `null`, `similarity_status = "insufficient_history"`, `history_transactions` gives the count. It is **not** a fraud probability and not a model confidence. Experiment 3: ROC-AUC 0.866 on synthetic data, no added value over the classifier (Chapter 13)
Alert levels	Low < 25 ≤ Medium < 50 ≤ High < 80 ≤ Critical on the Fraud Score — legacy DNN-era bands (R-02)
Calibration	none in the serving path: scores are model scores, not probabilities. Experiment 3 measured calibration offline (Chapter 13); nothing was adopted

Interaction. Features → LSTM → Risk Score → (with the nine features) random forest → Fraud Score → SHAP and alert level; similarity is computed independently from the nine features alone (and only from 10 earlier transactions). **Cold start.** With fewer than ten earlier transactions the LSTM is skipped and Risk Score is the training mean; with none, baseline-relative features are imputed. **Model sets.** `v2_lstm_rf_seed14` (default), `production` (v1 LSTM + DNN), `v2_dnn_lstm`, `v2_dnn_only`, `v2_dnn_lstm_seed14`; an unknown `MODEL_SET` stops start-up. The default and seed-14 sets are verified by SHA-256 *before* the pickled forest is loaded.

8. API and database

Endpoints (Appendix A lists them all): `POST /predict`, `POST /predict/batch`, `POST /report/pdf`, `GET /customers`, `GET /customer/{id}/profile`, `GET /customer/{id}/history`, `GET /fraud-rings`, `GET /model-info`, `GET /metrics`, `GET /metrics/report`, `GET /health`. Interactive documentation is served at `/docs` (set `API_DOCS=off` to remove `/docs`, `/redoc` and `/openapi.json`).

Request (`POST /predict`) — fields and limits after the audit fixes:

```
{ "customer_id": "CUST_0001",        // 1-64 characters, trimmed
  "amount": 85000,                    // > 0 and <= 1,000,000,000, finite
  "merchant_category": "electronics", // 1-64 characters
  "device_id": "DEV_UNKNOWN_9999",    // 1-64 characters
  "location": "Lagos",                // 1-64 characters
  "failed_logins_24h": 4,             // 0 .. 10,000
  "timestamp": "2026-07-10T13:30:00Z" // optional; aware values converted to naive UTC; must lie in
2000-01-01 .. 2100-01-01
}
```

Response (abridged, from a real run with the README example): `transaction_id`, `customer_id`, `timestamp`, the echoed inputs, `risk_score` (15.49), `fraud_probability` (89.46 — the field name is historical; it is the Fraud Score, not a probability), `alert_level` ("Critical Risk"), `similarity_pct` (0.26), `deviation_pct`, `similarity_status` ("ok" or "insufficient_history"), `history_transactions`, and `reasons` (Foreign Location, New Device, Multiple Failed Logins, Amount Far Above Average, each with `feature`, `display_name`, `shap_value`).

Compatibility notes (follow-up). `similarity_pct` and `deviation_pct` may be `null` (clients must accept it); `POST /report/pdf` now takes only `{"transaction_id": "…"}` (1–64 characters of letters, digits, `_ . -`), reads the stored transaction and prints stored values: extra fields are ignored, a malformed id is 422 and an unknown id is 404.

Errors: 422 validation (body `{"detail": [{type, loc, msg, input}]}`), 400 bad CSV, 404 no history or unknown transaction (report), 413 batch too large or declared request body too large, 503 `/health` when the database is unreachable, 500 only for unexpected faults.

Database. One table, `transactions`: `transaction_id` (PK), `customer_id` (indexed, not null), `timestamp`, `amount`, `merchant_category`, `device_id`, `location`, `failed_logins_24h`, `risk_score`, `fraud_probability`, `similarity_pct`, `deviation_pct` (both nullable since the follow-up), `alert_level`, `is_fraud_actual` (unused, nullable), `model_set`, `model_version` (provenance; NULL for older rows), `reasons_json` (the stored SHAP reasons used by the PDF; NULL for rows saved before it existed). Tables are created at start-up; columns added later are applied by `ensure_schema` (`ALTER TABLE … ADD COLUMN`). At start-up the saved rows are replayed into the in-memory histories. No foreign keys, no uniqueness beyond the primary key, no migration framework (R-10).

9. Frontend and user workflow

The single-page dashboard has four tabs. **Live Scan:** choose a customer (profile with usual device and city), pick the *Typical Purchase* or *Suspicious Pattern* scenario or enter values, press *Scan Transaction*; the page shows the verdict banner (alert level and the band that produced it), three gauges (Risk Score, Fraud Score, Behavioral Similarity), the SHAP reasons chart, a baseline-vs-transaction table, the Fraud Evolution Timeline of scored transactions, and a *Download Report* button (the dashboard sends only the transaction id; the report is built on the server from the stored row). **Batch Scoring:** upload a CSV (required `customer_id, amount, merchant_category`; optional `device_id, location, failed_logins_24h`), see the alert distribution and a filterable table. **Fraud Rings:** customers who share a device id. **Model Performance:** the loaded model set's metadata and evaluation. All requests go through `src/api.js`; the footer and header state the model set and that scores are not calibrated probabilities.

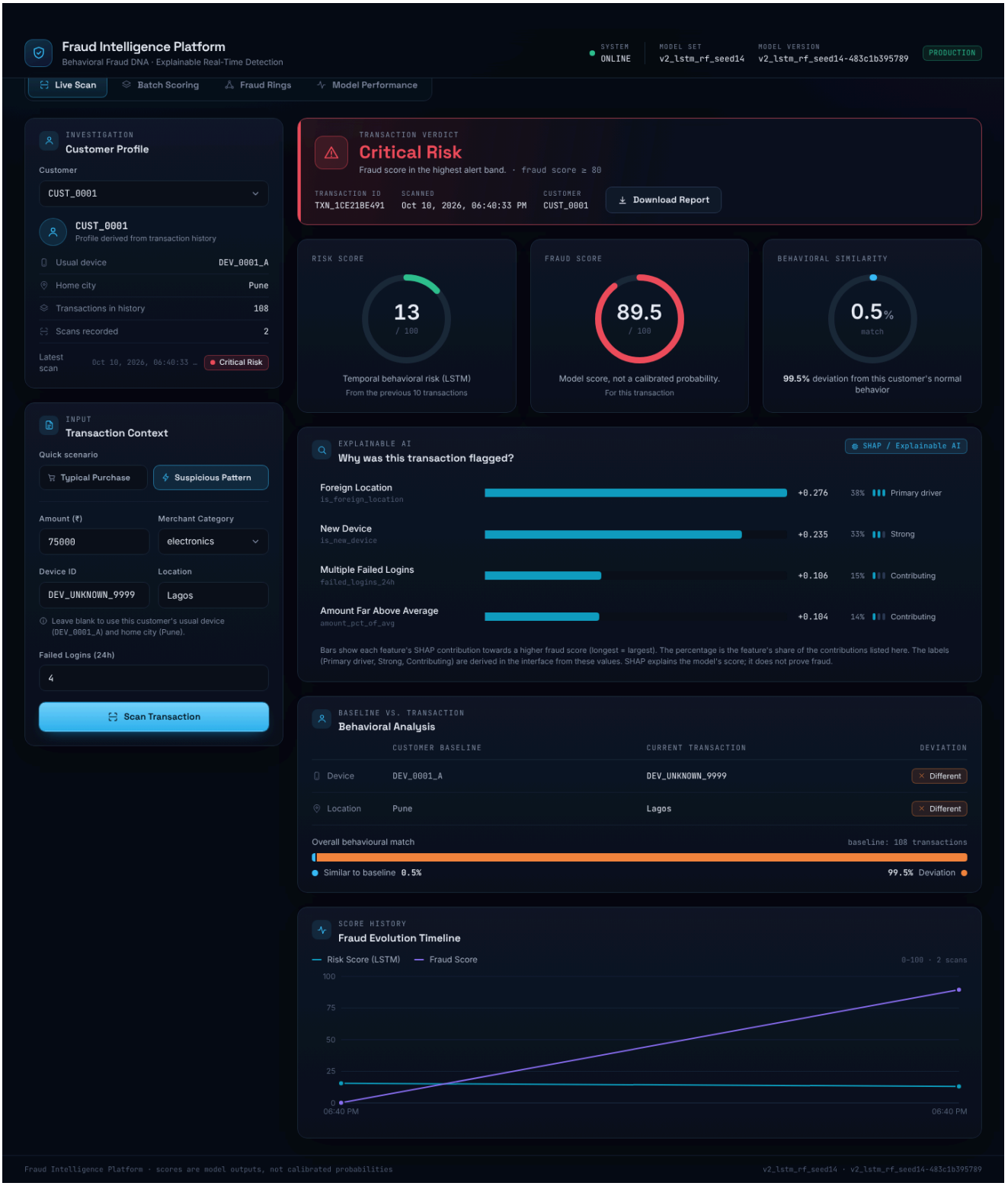


Figure 2. Live Scan after the "Suspicious Pattern" scenario (real run, default model set): Critical Risk, Fraud Score 89.5, four SHAP reasons.

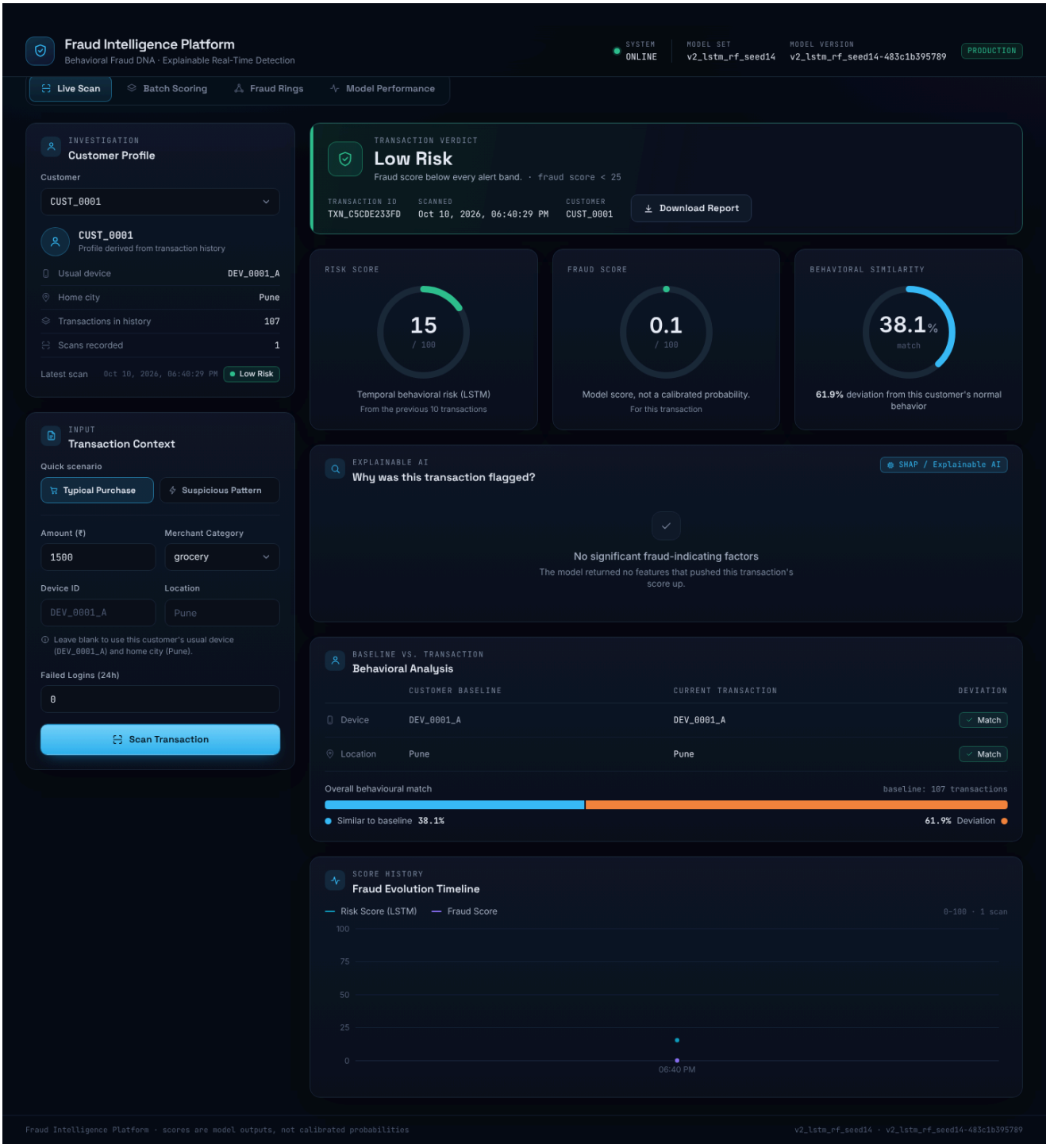


Figure 3. The "Typical Purchase" scenario: Low Risk, Fraud Score 0.1, no reasons.

Insufficient history. When `similarity_status` is `insufficient_history` the similarity card says "Not enough history" with the number of earlier transactions, shows no ring, number or bar, and the behavioural card shows a matching note; API validation errors appear as readable field messages.

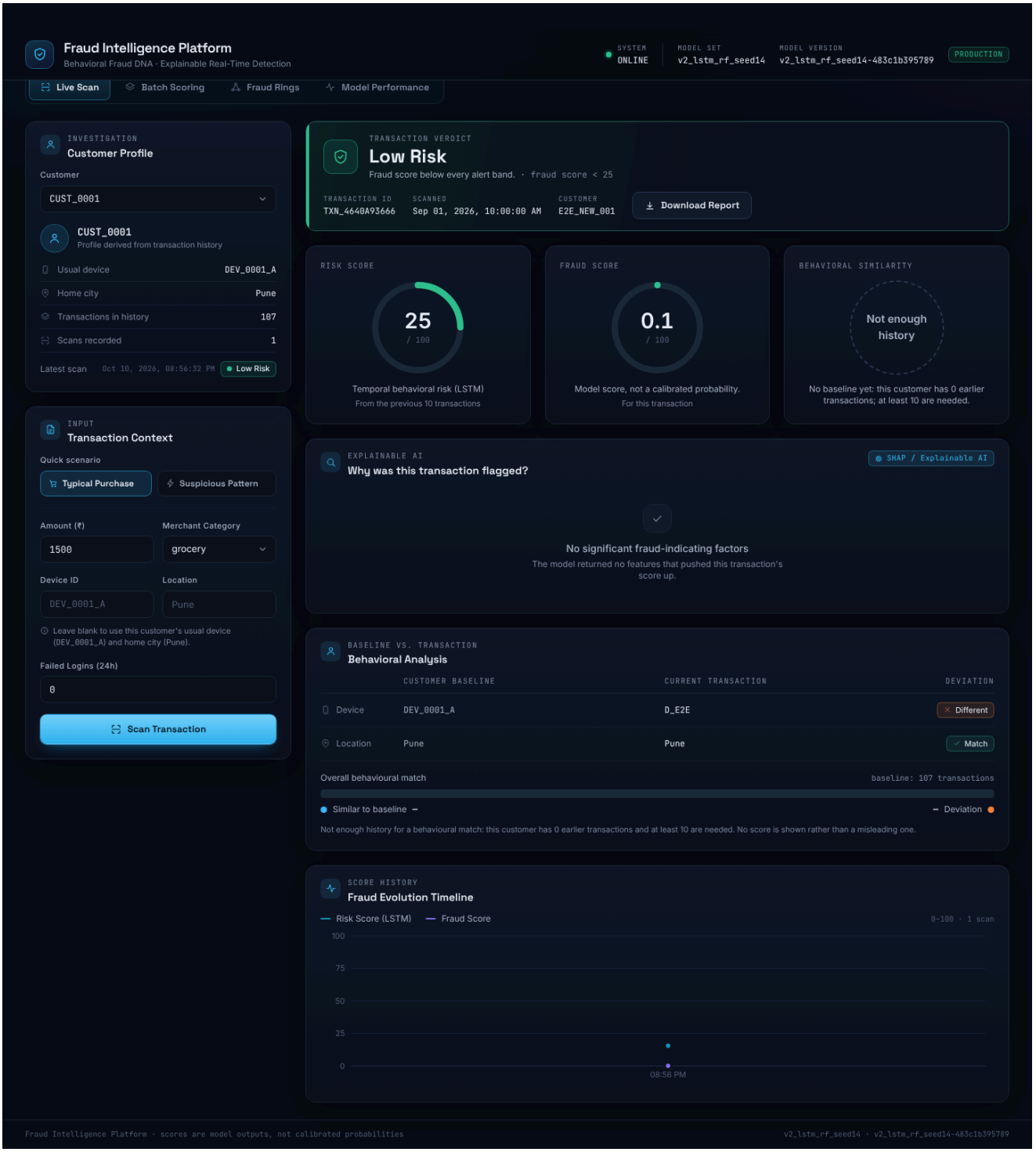


Figure 3b. The "Not enough history" state, rendered by the real dashboard from the real API response for a new customer. The dashboard has no way to create a customer, so that response was supplied to the page by request interception; the left panel still shows the selected customer's profile.

Field mapping verified. `risk_score` → Risk gauge, `fraud_probability` → Fraud Score gauge, `similarity_pct` / `deviation_pct` → similarity gauge and behavioural-match bar, `reasons` → SHAP chart, `alert_level` → banner. The gauge tint reuses the 25/50/80 bands (R-02, R-11).

10. Setup and execution

Commands below were executed in this audit unless marked *(not executed)*.

```
# prerequisites: Python 3.12+ (3.13 used), Node 18+ (22 used)
cd backend
python3 -m venv venv && source venv/bin/activate           # Windows: venv\Scripts\Activate.ps1
pip install -r requirements.txt -r requirements-dev.txt    # ~2.5 min, 75 packages
uvicorn app.main:app --port 8000                           # add --reload for development
curl -s localhost:8000/health                              # {"status":"ok"}  (503 if the database is unreachable)
```

```
cd frontend
npm ci                          # or npm install
npm run lint                    # oxlint, exit 0
npm run build                   # production build to frontend/dist
npx vite preview --port 4173    # serves the build; /api is proxied to localhost:8000
npm run dev                     # development server on 5173 (not executed in this audit)
```

```
cd backend && pytest -q                # 707 passed, 4 skipped in ~14 min on 4 cores (follow-up run)
pytest -m "not slow"                   # (not executed) skips the slowest tests
pytest tests/test_audit_fixes.py tests/test_similarity_history.py tests/test_security_defaults.py   # regression tests of both passes (seconds)
cd ../frontend && npm test             # 13 vitest tests (static rendering), < 1 s
```

Configuration (environment variables): `MODEL_SET` (default `v2_lstm_rf_seed14`), `DATABASE_URL` (default SQLite file in `backend/`), `CORS_ALLOW_ORIGINS`, `BATCH_MAX_BYTES` (default 2,000,000), `BATCH_MAX_ROWS` (default 500), `API_DOCS=off` (hide interactive docs). Use a throw-away `DATABASE_URL` for experiments; the default file is git-ignored.

Troubleshooting. *Frontend "Could not reach the backend":* check `curl localhost:8000/health`. *"Address already in use":* use another port and update `vite.config.js`. *`ModuleNotFoundError` in training scripts:* run `python -m app.…` from `backend/`. *Batch rejected with 413:* the file exceeds the limits above. *A request fails with 422:* read `detail`; values outside the documented ranges are rejected. Regenerating data and retraining (README §5) was **not executed** here.

11. Audit findings

The full register (evidence, root cause, consequence, fix, verification, limitations) is `reports/BUG_AND_RISK_REGISTER.md`. Summary:

ID	Pri	Finding	Status
F-01	P1	`amount = Infinity` accepted and stored; customer's features become NaN permanently	fixed
F-02	P2	`amount = NaN` → HTTP 500	fixed
F-03	P2	blank / 100,000-character identifiers accepted (phantom customers)	fixed
F-04	P2	timezone-aware timestamp → HTTP 500	fixed
F-05	P2	in-memory history can get ahead of the database when a commit fails	fixed
F-08	P2	batch upload without size or row limit	fixed
F-06	P3	unbounded `limit` / `min_customers` (`limit=-1` drops a row)	fixed
F-07	P3	PDF filename header injection; non-Latin-1 id → 500	fixed
F-09	P3	`/health` always "ok"	fixed

ID	Pri	Finding	Status
F-10	P3	axios and source-map-js advisories (2 high)	fixed
F-11	P3	URL path segments not encoded	fixed
R-01	P1*	no authentication or authorisation (*if exposed beyond localhost*)	open, decision
R-02	P2	alert bands are legacy DNN bands applied to the forest's scores	open, decision
R-03	P2	per-request cost grows with history length (6.1 s at 3,200 rows)	open
R-04	P2	PDF report printed client-supplied scores (forgeable)	**fixed** (id-only request; stored values)
R-05	P2	similarity meaningless for customers without history (1.9 % for an ordinary first purchase)	**fixed** (`null` + status below 10 earlier transactions)
R-06	P3	unbounded timestamps	**fixed** (2000–2100)
R-09	P3	v1 generator not reproducible	**fixed** (fixed start date and seed)
R-11, R-12	P3	frontend error text and size; the frontend had no tests	**partly fixed** (readable errors; 13 tests; chunk warning remains)
R-07, R-08, R-10	P3	unpinned `production` artifacts; v1 feature-version drift; hygiene	open
R-13	P3	transactions stored before the follow-up have no stored explanation for PDFs	open, documented
R-14	P3	similarity formula dominated by near-constant features (94.5 % of rows)	open, measured (Experiment 3)
R-15	P3	the dashboard cannot create a customer; "not enough history" UI verified by response injection	open

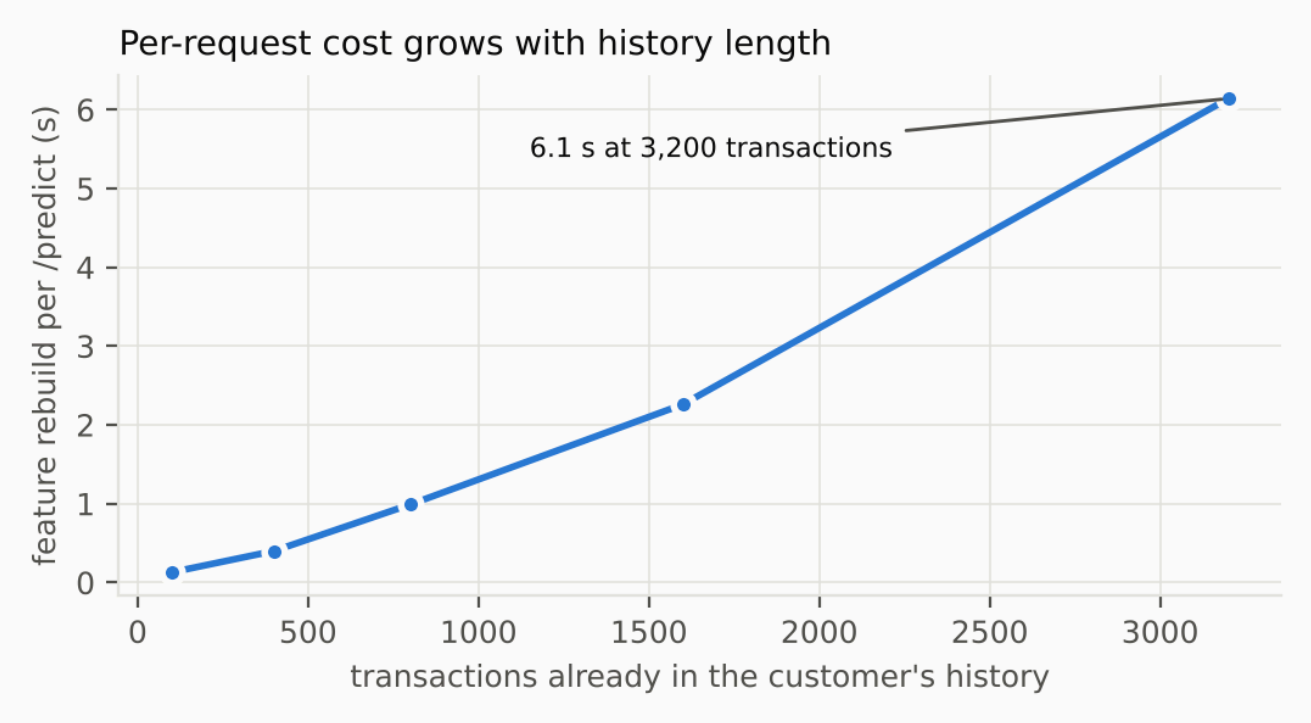


Figure 4. Measured cost of rebuilding one customer's features on each request (R-03). Single runs with other load present; indicative.

12. Completed fixes

Fix	Files	Test evidence
Input validation on `/predict` (finite and bounded amount, bounded logins, 1–64 character trimmed identifiers, timezone normalisation)	`schemas.py`	22 tests (incl. 3 positive controls); the rest failed on the old code
Validation errors that echo NaN no longer 500	`main.py`	1 of the above
Roll back in-memory history when persistence fails (single and batch)	`inference_pipeline.py`, `main.py`	4 tests
Bounded query parameters	`main.py`	6 tests
Safe download filename	`main.py`, `frontend/src/api.js`	4 tests
Batch limits and parity of field limits	`main.py`	3 tests
Health check verifies the database	`main.py`	2 tests
axios upgrade, URL encoding	`frontend/package*.json`, `src/api.js`	`npm audit` 0; lint and build pass; browser run
Follow-up: similarity withheld below 10 earlier transactions (R-05)	`similarity.py`, `inference_pipeline.py`, `schemas.py`, `report.py`, `SimilarityMeter.jsx`, `BehavioralAnalysis.jsx`	`test_similarity_history.py` (0, 1, 9 and 10, 50 earlier transactions); 13 frontend tests
Follow-up: PDF from stored transaction (R-04)	`main.py`, `schemas.py` (`ReportRequest`), `db/models.py` (`reasons_json`), `db/migrations.py`, `report.py`, `api.js`	`test_report.py` tamper, 404, 422 and null-reasons tests
Follow-up: timestamp window (R-06)	`schemas.py`	4 boundary tests
Follow-up: security defaults (CORS limits, nosniff, no-store, length guard, `API_DOCS`)	`main.py`	`test_security_defaults.py` (9 tests)
Follow-up: reproducible v1 generator (R-09), readable API errors (R-11)	`data/generate_synthetic_data.py`, `frontend/src/apiError.js`	`test_v1_generator.py`; frontend tests

Tested: first pass: 41 new tests (38 fail on the original code, 41 pass now), full suite 652 passed, 4 skipped, 0 failed. Follow-up: the new and changed test modules give **33 failures and 1 collection error on the original code** and pass on the fixed code; full suite **707 passed, 4 skipped, 0 failed**; frontend 13 tests, lint exit 0, build, `npm audit` 0, `pip-audit -r requirements.txt` clean; browser runs (`reports/evidence/`). **Not independently tested:** a crash between scoring and rollback is not covered by F-05; a browser click-through of a new customer (the dashboard cannot create one; the state was verified with the real API response injected); the 4 skipped backend tests.

13. Experimental results

Only results that exist in committed files and could be cross-checked are reported. Details and gaps: `reports/ML_EXPERIMENT_STATUS.md`.

Independent re-computation (this audit). From the committed per-row scores of the v2 time-split evaluation:

Test period: 21,142 windows, 61 fraud rows (from the committed `scores_time_split.csv.gz`).

Model	PR-AUC recomputed	PR-AUC in report	ROC-AUC recomputed	ROC-AUC in report
lstm_risk_predictor	0.1345	0.1345	0.7407	0.7407
dnn_fraud_classifier	0.1045	0.1045	0.8993	0.8993
amount_hour_rule	0.0272	0.0272	0.7296	0.7296
logistic_regression	0.1375	0.1375	0.8905	0.8905
dnn_without_risk_score	0.1283	0.1283	0.8965	0.8965

Fresh hold-out, seeds 501–505 (single-use, pre-registered protocol "4D v1"; values copied by script from `final_holdout_report.json`):

5 fresh datasets, 487,368 transactions, 2,390 fraud transactions, 550 fraud episodes, 550 first-fraud transactions.

Model	PR-AUC [95% CI]	ROC-AUC	Recall [95% CI]	Precision	F1	Legit alerts / 1,000	First-fraud recall [95% CI]	Episodes detected
LSTM + random forest (default)	0.464 [0.419, 0.509]	0.939	0.645 [0.608, 0.683]	0.236	0.346	10.23	0.738 [0.699, 0.776]	439 / 550
LSTM + DNN (seed 14)	0.328 [0.286, 0.371]	0.929	0.548 [0.505, 0.588]	0.217	0.311	9.68	0.595 [0.550, 0.637]	386 / 550
Previous production (v1 LSTM + DNN)	0.133 [0.111, 0.157]	0.691	0.255 [0.228, 0.283]	0.104	0.148	10.71	0.384 [0.340, 0.427]	322 / 550

New-customer variant, seeds 511–515:

5 fresh datasets, 450,095 transactions, 2,693 fraud transactions, 632 fraud episodes, 549 first-fraud transactions.

Model	PR-AUC [95% CI]	ROC-AUC	Recall [95% CI]	Precision	F1	Legit alerts / 1,000	First-fraud recall [95% CI]	Episodes detected
LSTM + random forest (default)	0.490 [0.449, 0.531]	0.939	0.638 [0.603, 0.675]	0.287	0.396	9.50	0.738 [0.696, 0.776]	492 / 632
LSTM + DNN (seed 14)	0.365 [0.323, 0.408]	0.928	0.533 [0.495, 0.572]	0.266	0.355	8.78	0.599 [0.555, 0.642]	440 / 632
Previous production (v1 LSTM + DNN)	0.163 [0.138, 0.188]	0.705	0.269 [0.241, 0.297]	0.133	0.178	10.49	0.399 [0.355, 0.443]	366 / 632

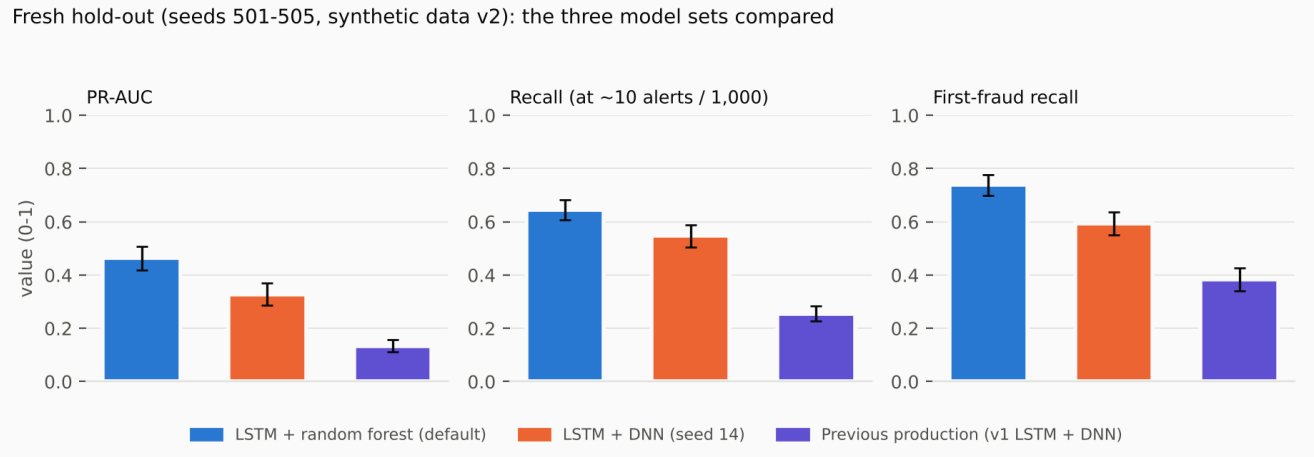


Figure 5. Hold-out comparison with 95 % confidence intervals (error bars).

Counts and classes. The v1 seed has 0.872 % fraud rows; the default v2 dataset 0.465 %; the hold-outs 0.49 % (2,390 / 487,368); the Experiment 3 test seeds 0.47 % (2,910 / 620,877). **Baselines** (v2 time-split): an amount-and-hour rule has PR-AUC 0.027; logistic regression 0.1375 and the DNN without the LSTM score 0.1283 are at or above the DNN with it (0.1045). **Interpretation limits:** synthetic data from one generator; 61 fraud rows in the v2 time-split test period; the serving score is not calibrated; the similarity score and calibration were evaluated only on synthetic data (below); no real-world validation of any kind.

Experiment 3 (follow-up): similarity, calibration and ablations on fresh synthetic seeds

Protocol `experiments/validation_exp3/PROTOCOL.md` was committed (`b58fa52`, amendment `e81b356`) before any data was generated. Validation seeds 9301–9302 and test seeds 9303–9305, 1,000 customers each (test: 620,877 rows, 2,910 fraud rows, 660 episodes; every dataset matched its manifest SHA-256, listed in `experiments/validation_exp3/RESULTS.md`). Anything fitted or chosen used the validation seeds only; the test seeds were scored once. Intervals: 95 % customer-component bootstrap. The headline numbers were recomputed independently from the per-row scores (51 of 51 agree) and a second complete run gave identical results. **Synthetic data, one generator.**

Behavioral Similarity Score (rows with at least 10 earlier transactions; prevalence 0.0049):

Score	ROC-AUC [95 % CI]	PR-AUC [95 % CI]
similarity deviation (served formula)	0.866 [0.852, 0.878]	0.069 [0.059, 0.081]
variant: std floored at 0.5	0.859 [0.846, 0.872]	0.044 [0.038, 0.050]
default classifier score	0.941 [0.932, 0.950]	0.507 [0.466, 0.547]
classifier + similarity	0.941 [0.932, 0.950]	0.508 [0.467, 0.549]

Fraud type (against all legitimate rows)	Fraud rows	ROC-AUC [95 % CI]
account takeover	647	0.895 [0.877, 0.913]
card testing cashout	714	0.948 [0.937, 0.958]
device takeover	159	0.929 [0.904, 0.950]
high value single	105	0.989 [0.984, 0.993]
normal looking	165	0.618 [0.569, 0.670]
ring	262	0.972 [0.960, 0.982]
stolen card online	858	0.763 [0.737, 0.790]

It separates fraud from legitimate rows (median deviation 11.5 versus 30.2), best for high-value single, ring and card-testing fraud and worst for "normal-looking" fraud. Combined with the classifier it changes PR-AUC by +0.0011 [−0.0002, +0.0023]: **no added information is shown**. It correlates strongly with the classifier within fraud rows (Spearman 0.77). In 94.5 % of rows at least one feature has zero spread in the customer's history (R-14); flooring that spread made the score worse. Because fraud in this generator departs from the customer baseline on the same features, a high AUC shows the score reflects the generator's rules, not that it finds real fraud.

Calibration of the Fraud Score (default model, all test rows; calibrators fitted on validation only):

Version	Brier	Log loss	ECE (equal mass)	Slope	Brier difference vs none [95 % CI]
uncalibrated	0.003295	0.01609	0.00049	1.10	–
sigmoid	0.003250	0.01601	0.00021	1.05	-0.000044 [-0.000061, -0.000028]
isotonic	0.003159	0.01575	0.00016	1.03	-0.000135 [-0.000169, -0.000101]

The raw score is already close to a probability on this generator (mean predicted 0.0044 versus observed 0.0047; the top bin of 41,391 rows: predicted 0.051, observed 0.056). Both calibrators lower the Brier score and log loss with intervals excluding 0 and in every test seed, but only by about 1–4 % of the Brier value; isotonic reduces the score to 155 distinct values. Nothing was changed in serving; the score is still described as a model score.

Baselines and ablations (identical rows, cut-off fixed on validation at a 1 % false-positive rate, test seeds pooled):

Model	PR-AUC [95 % CI]	ROC-AUC	Recall	Precision	Legit. alerts per 1,000 legit.
default (LSTM -> random forest)	0.507 [0.466, 0.547]	0.941	0.642	0.251	9.48
LSTM -> DNN	0.373 [0.331, 0.413]	0.932	0.558	0.220	9.80
DNN only (no LSTM)	0.209 [0.177, 0.242]	0.893	0.405	0.173	9.57
production v1 (LSTM -> DNN, different training data)	0.172 [0.147, 0.198]	0.709	0.279	0.125	9.71
random forest, no LSTM input	0.424 [0.383, 0.464]	0.910	0.560	0.224	9.62
logistic regression (balanced)	0.289 [0.253, 0.325]	0.891	0.411	0.173	9.72
amount-and-hour rule	0.032 [0.028, 0.038]	0.666	0.201	0.089	10.23
similarity deviation alone (S0)	0.069 [0.059, 0.081]	0.866	0.238	0.102	10.39
default + similarity (logistic combination)	0.508 [0.467, 0.549]	0.941	0.638	0.248	9.55

The LSTM input adds detectable value over the same random forest without it (PR-AUC +0.083 [0.069, 0.096], recall +0.081 [0.066, 0.096] at the same false-alert rate), and the random forest beats the DNN variants and the simple baselines. At about 9.5 false alerts per 1,000 legitimate transactions the default model's precision is 0.25. On the 30,000 test rows with fewer than 10 earlier transactions (all legitimate) the default model alerts on 5.9 % at the same cut-off: the known new-customer weakness, measured but not remedied. **Comparison with the earlier hold-out (seeds 501–505):** the ranking of the default, LSTM → DNN and production models is the same (PR-AUC 0.507

/ 0.373 / 0.172 here versus 0.464 / 0.328 / 0.133 there) and the default model's intervals overlap; only ranking metrics are compared because the datasets are different, larger draws and the cut-off rules differ. This is a consistency check on one generator, not independent confirmation.

Independent data. None exists in the project and none was downloaded. `experiments/validation_exp3/INDEPENDENT_VALIDATION_STATUS.md` states the required dataset characteristics, feature mapping, leakage risks and the final-evaluation procedure.

14. Security and reliability

This was a defensive review of the local project and its test environment only; it was **not** a penetration test or a formal certification.

Area	Finding
Authentication / authorisation	none (R-01). Every endpoint, including history, rings, batch and reports, is open. Acceptable for a local demo, not for a network. Requirements and three options (gateway SSO, bearer tokens, API key — unsuitable for the dashboard) are in `docs/deployment-security-requirements.md`; **no scheme was chosen or built**
Input validation	weak on `/predict` before the audit (F-01 … F-04); now bounded. Batch rows follow the same limits
Injection	SQL: SQLAlchemy ORM with bound parameters, no raw user SQL found. XSS: React escapes text; no `dangerouslySetInnerHTML` was used in the files read. Header injection through the PDF filename: fixed (F-07)
Abuse resistance	no rate limiting; batch capped at 2 MB / 500 rows (F-08); a declared `Content-Length` above the batch limit plus 100 kB gets 413 (a chunked body is not counted, so the proxy must enforce size). The un-merged `keerthan` branch has a limiter (not evaluated)
CORS	explicit localhost allow-list by default; wildcard disables credentials; methods limited to GET/POST/OPTIONS and headers to `Content-Type`, `Accept`
Response headers	`X-Content-Type-Options: nosniff` on every response; `Cache-Control: no-store` on API responses
Report integrity	PDFs are built from the stored transaction; a client cannot alter printed values (R-04 fixed)
Secrets	none tracked; only `.env.example` templates; `.env` and `*.db` are git-ignored
Dependencies	backend: no known vulnerabilities (`pip-audit` on the 69 installed packages in the first pass and on `requirements.txt` in the follow-up); frontend: 0 (`npm audit`)
Model integrity	default and seed-14 artifacts hashed before deserialisation; `production` set not pinned (R-07)
Error disclosure	500 responses are generic; debug mode is not enabled; `/docs` is public unless `API_DOCS=off`
Reliability	memory/database drift closed (F-05); health check verifies the database (F-09); single-process in-memory state; start-up replays all rows; cost grows with history (R-03)
Privacy	all data synthetic; real deployment would need a retention policy and access control for the `transactions` table

15. Limitations and future improvements

Technical. Single process with in-memory histories (several workers would diverge); per-request recomputation; naive timestamps; SQLite default with PostgreSQL untested; no migration framework; frontend tests cover only the new states (13 tests). **Scientific.** One synthetic generator; trivially separable v1 data; small v2 time-split test; hold-out is single-use and from the same generator; no calibration in serving; the LSTM is not shown to warn before a first fraud; the similarity score and calibration were evaluated only on synthetic data (Experiment 3), where the similarity adds nothing

beyond the classifier. **Prioritised improvements** (`reports/RECOMMENDED_IMPROVEMENTS.md`): (1) decide the alert definition and align bands with the forest's validated cut-offs; (2) add authentication, TLS and rate limiting; (3) incremental features; (4) obtain independent data and re-measure calibration there before changing the score's meaning; (5) extend frontend tests and add monitoring. Done in the follow-up: "not enough history" for similarity, trusted reports.

16. Final project status

Working functionality	scoring with explanation, similarity, persistence and replay, history timeline, batch scoring, PDF report, fraud rings, model-performance view, model-set switching with integrity checks
Fixed defects	11 (F-01 … F-11) plus four open items fixed (R-04, R-05, R-06, R-09) and two partly fixed (R-11, R-12)
Tests passed	backend **707 passed, 4 skipped**; frontend 13 passed; `oxlint` exit 0; production build; `npm audit` 0; `pip-audit` clean; browser end-to-end runs; Experiment 3 recomputation 51 of 51
Tests failed	none at the current revision (the failures recorded in the fix log are the new tests run against the *original* code, by design)
Tests not executed	4 skipped backend tests (need `data/v2` CSVs); hold-out re-run; retraining and the project's own evaluation scripts; PostgreSQL; `npm run dev`; a new-customer click-through (the UI has none); any load test beyond the project's own
Remaining bugs / risks	9 open items (R-01, R-02, R-03, R-07, R-08, R-10, R-13, R-14, R-15) and 2 partly fixed (R-11, R-12)
Security concerns	no authentication or authorisation; no rate limiting; no TLS; length guard sees only declared lengths; `/docs` public unless switched off
Missing evidence	any real or independent dataset (the main gap); calibration and similarity results on anything but the project's own generator; early-history fraud (none in the Experiment 3 data)
Next actions	owner decisions (alert bands, authentication scheme, calibration in serving, independent data), then Chapter 15; the IEEE paper has not been started

---

Appendix A. API endpoint inventory

Method and path	Purpose	Notes
`POST /predict`	score one transaction	body above; persists the result; 422 on invalid input; similarity fields may be `null`
`POST /predict/batch`	score a CSV (multipart `file`)	all-or-nothing; ≤ 2 MB and 500 rows (413); 400 lists invalid rows (first 20)
`POST /report/pdf`	PDF for one stored transaction	body `{"transaction_id": …}` only; every printed value comes from the stored row; 422 malformed id, 404 unknown id
`GET /customers?limit=`	known customer ids	`limit` 1–10,000, default 50
`GET /customer/{id}/profile`	usual device and city	404 without history
`GET /customer/{id}/history?limit=`	scored timeline, newest first	`limit` 1–1,000; 404 if none scored
`GET /fraud-rings?min_customers=`	devices shared by several customers	`min_customers` ≥ 1, default 2
`GET /model-info`	metadata of the loaded model set	versions, thresholds, evaluation pointers

Method and path	Purpose	Notes
`GET /metrics`, `GET /metrics/report`	evaluation of the loaded model set	content depends on the model set
`GET /health`	liveness and database check	200 `{"status":"ok"}` or 503

Appendix B. Feature dictionary

See Chapter 6. Model input order: `amount_zscore, hour_is_unusual, is_new_device, is_new_location, is_foreign_location, failed_logins_24h, category_is_unusual, txn_velocity_1h, amount_pct_of_avg, risk_score`.

Appendix C. Model configuration

Item	Value
Sequence length	10
LSTM	64 → 32 units, dropout 0.3/0.2, dense 16, sigmoid; 31,905 parameters
Random forest	200 trees, max depth 12, min samples per leaf 5, no class weights, 10 inputs
DNN	64 → 32 → 16 → 1; 3,585 parameters
Input scaling	saved means/stds; clipped to ±6
Output	Fraud Score = forest score × 100, capped at 99.9
Reason display	SHAP top 4 positive, only if score ≥ 5
Policy cut-offs (default set, validation-chosen)	alert budget score 4.39; critical 26.2 (recorded, not used for alert levels)
Seed / training data	seed 14; synthetic v2 dataset (SHA-256 `a09d0611…b1ef08`), 58,840 training rows, 291 fraud

Appendix D. Test inventory

At the current revision: 37 modules and 711 tests collected; the last full run gave 707 passed and 4 skipped, 0 failed.

Test module	Tests
`test_audit_fixes.py`	42
`test_backdated.py`	10
`test_basic_endpoints.py`	5
`test_batch.py`	35
`test_candidates.py`	14
`test_cold_start.py`	27
`test_cold_start_sensitivity.py`	28
`test_compare_candidates.py`	9
`test_concurrency.py`	3
`test_cors.py`	13

Test module	Tests
`test_dataset_versions.py`	25
`test_downstream_selection.py`	26
`test_evaluation.py`	24
`test_final_holdout.py`	13
`test_fraud_rings.py`	7
`test_history.py`	10
`test_holdout.py`	32
`test_home_device.py`	17
`test_metrics.py`	14
`test_model_sets.py`	60
`test_multiseed.py`	22
`test_new_customer_generator.py`	12
`test_observability.py`	27
`test_persistence.py`	11
`test_predict.py`	22
`test_profile.py`	9
`test_report.py`	17
`test_rf_model_set.py`	25
`test_scoring_units.py`	15
`test_security_defaults.py`	19
`test_seed14_model_set.py`	30
`test_selection.py`	29
`test_similarity_history.py`	14
`test_synthetic_v2.py`	20
`test_synthetic_v2_sanity.py`	6
`test_v1_generator.py`	9
`test_v2_analysis.py`	10
Total	**711**

Appendix E. Glossary

PR-AUC area under the precision–recall curve (suited to rare positives). **ROC-AUC** area under the ROC curve. **Recall** share of fraud found. **Precision** share of alerts that are fraud. **First-fraud recall** share of episodes whose first fraudulent transaction was alerted. **FPR** false-positive rate. **SHAP** per-feature contribution to a model's score. **Out-of-fold** a score produced by a model that did not train on that row. **Cold start** a customer with fewer than ten earlier transactions. **Episode** a customer's connected fraud transactions. **Hold-out** data generated and scored once for confirmation. **Synthetic data** data produced by a program, not by real customers.